

QuickRAG: Production Cloud-Native RAG Platform

System Architecture, Code Review, Trade-off Analysis & Deployment Report

Live Hugging Face Space: <https://huggingface.co/spaces/shresthiwari/QuickRAG>

Live Render FastAPI Backend: <https://quickrag.onrender.com>

1. Interviewer Pitch & Technical Defense Script

Realistic 60-Second Elevator Pitch (Zero Fluff, Technical Focus):

"QuickRAG is an enterprise-grade Retrieval-Augmented Generation platform designed to eliminate LLM hallucinations and contextual boundaries when querying multi-format document clusters. The system decouples document processing from LLM synthesis using a cloud-native architecture: FastAPI serves async endpoints, document embeddings are dynamically chunked using RecursiveCharacterTextSplitter and vectorized with Mistral AI (1024 dimensions), and vectors are stored in Pinecone Cloud for scalable ANN similarity search. On the front end, a Gradio interface allows users to perform multi-file bulk uploads and interactive knowledge retrieval. I specifically engineered rate-limit safe batch throttling for vector upserts and implemented deterministic system prompts to enforce strict context-grounded responses."

Anticipated Interviewer Cross-Questions & Instant Technical Responses:

Q1: How do you prevent LLM hallucinations when context is missing?

We enforce strict prompt grounding using system directives ("Use ONLY provided context. If missing, respond: I could not find the answer in the document"). Temperature is kept low, and context retrieval defaults to top k=7 cosine similarity chunks.

Q2: Why use RecursiveCharacterTextSplitter over simple CharacterSplitter?

Simple CharacterSplitter splits blindly on character length, often truncating sentences mid-word or breaking logical paragraphs. Recursive character splitting tries ["\n\n", "\n", " ", ""] in order, preserving structural paragraph and sentence boundary semantics.

Q3: How do you handle API rate limits during bulk document ingestion?

In vector store/DB.py, we implemented chunk-level batching (batch_size=50) with a 3-second sleep delay between API calls to prevent HTTP 429 exceptions on API free tiers.

Q4: Why select Mistral AI (mistral-embed / mistral-small) over OpenAI?

Mistral provides top-tier 1024-dimensional dense embeddings with open-weights efficiency, significantly lower latency and cost per 1M tokens compared to OpenAI text-embedding-ada-002, while offering superior instruction compliance for precise context lookup.

2. Project Architecture & File-by-File Walkthrough

The project follows a modular 4-tier RAG pipeline: **Ingestion Tier** (PDF/Text extraction) → **Vectorization Tier** (Text chunking & Mistral Embeddings) → **Retrieval Tier** (Pinecone Cloud Vector Search) → **Synthesis Tier** (FastAPI / Gradio / Streamlit + Mistral LLM).

File / Module	Role & Responsibility	Key Technologies
main.py	Production FastAPI backend deployed on Render Cloud (https://quickrag.onrender.com). Serves /api/chat and /api/upload endpoints.	FastAPI, Pydantic, PineconeVectorStore, MistralAI
app.py	Local Streamlit UI demonstration with PDF drag-and-drop, Chroma local vector database creation, and MMR retrieval.	Streamlit, Chroma DB, MistralAIEmbeddings
frontend_gradio.py	Multi-tenant web dashboard deployed on Hugging Face Spaces (https://huggingface.co/spaces/shresthtiwari/QuickRAG). Connects browser file uploads to Render REST API.	Gradio, Requests (Multipart Upload)
vector_store/DB.py	Standalone ingestion pipeline script. Scans document loaders/ directory, parses PDFs/TXTs, chunks, and uploads to Pinecone.	LangChain PyPDFLoader, TextLoader, Pinecone
create_database.py	Script for building local Chroma vector database persisted in chroma_db/.	Chroma, LangChain, MistralAIEmbeddings
retrievers/*	Exploratory retrieval modules showcasing Arxiv API fetching (arxiv.py), MMR diversity (mmr.py), and MultiQuery expansion (multiquery.py).	LangChain Retrievers, HuggingFaceEmbeddings

3. Technology Choices & Alternatives Analysis

FastAPI vs. Flask / Django: FastAPI was selected for native async endpoint support, automatic request payload validation via Pydantic, and OpenAPI spec generation. Flask lacks built-in async validation, while Django adds unnecessary ORM overhead for vector microservices.

Pinecone Cloud vs. Local Chroma DB: Pinecone provides zero-management cloud vector index persistence with sub-100ms ANN vector lookups across millions of vectors. Chroma was used for offline prototyping, but lacks serverless cluster auto-scaling required for multi-tenant production.

Mistral AI vs. OpenAI: Mistral's mistral-embed model delivers dense 1024-dimensional semantic embeddings with high accuracy and lower cost/latency than OpenAI. Combined with mistral-small-2506, it offers superior cost-to-performance ratio.

RecursiveCharacterTextSplitter vs. Fixed Chunking: Recursive splitting systematically maintains context integrity by honoring document structural hierarchies (paragraphs → sentences → words). Overlap of 200 characters prevents semantic loss across chunk boundaries.

4. Code Verification & Key Fixes Implemented

During project audit and verification, key code defects were identified and corrected:

- ✓ **Fix 1: FastAPI Control Flow & Endpoint Registration Order:** In `main.py`, the `/api/upload` endpoint definition was placed below `if __name__ == '__main__':` and contained duplicate imports. Moving all route definitions before uvicorn startup ensures proper endpoint binding upon import.
- ✓ **Fix 2: Vector Persistence in Ingestion Endpoint:** In `main.py`'s `/api/upload` route, document ingestion was parsing chunks but commenting out `vectorstore.add_texts(all_chunks)`. We enabled live vector upserts to Pinecone, allowing uploaded files to instantly update context.
- ✓ **Fix 3: Embedding Model Mismatch & API Key Consistency:** In `app.py` and `create_database.py`, `OpenAIEmbeddings()` was hardcoded, causing runtime authentication errors because `OPENAI_API_KEY` was missing from `.env`. We standardized all scripts to `MistralAIEmbeddings(model="mistral-embed")`.
- ✓ **Fix 4: API Throttling & Rate-Limit Safeguards:** In `vector_store/DB.py`, bulk chunk uploads were refactored into batch size of 50 chunks with a 3-second throttle delay, avoiding HTTP 429 Rate Limit errors during large PDF processing.

5. Core Project Questions & Technical Breakdown

• Why did you build it?

To engineer a production-ready, cloud-native RAG microservice that allows organizations to query unstructured PDF/TXT knowledge repositories with sub-second latency and zero hallucination.

• What problem does it solve?

It resolves LLM context token limitations and knowledge cutoffs by dynamically retrieving domain-specific document snippets and injecting them into generation prompts.

• Which technologies did you use?

Python 3.10+, FastAPI, LangChain, Pinecone Vector DB, Mistral AI (Embeddings & LLM), PyPDF, Pydantic, Gradio, Streamlit, and Python-dotenv.

• What challenges did you face?

Overcoming free-tier rate limits during bulk vector ingestion (solved via batching and sleep intervals), handling PDF text extraction edge cases, and coordinating REST API contract compatibility with Gradio multi-part file uploads.

• What would you improve?

Implement Hybrid Search (Sparse BM25 + Dense Vectors), Cohere Re-ranking for top-k precision, Pinecone Namespace isolation for per-user sessions, and asynchronous background ingestion using Celery and Redis.

6. Architectural Roadmap & Future Upgrades

- **Hybrid Vector Search:** Combine dense vector similarity (Mistral) with sparse keyword matching (BM25) via Reciprocal Rank Fusion (RRF) for specialized domain jargon.
- **Context Re-Ranking:** Integrate a re-ranker model (e.g., Cohere Rerank or BGE-Reranker) to evaluate retrieved candidates and pass only the top 3 high-confidence chunks to the LLM.
- **Asynchronous Worker Pipeline:** Offload heavy PDF parsing and vector calculation to Redis Task Queues and Celery workers to keep FastAPI endpoints non-blocking.
- **Multi-Tenant Session Isolation:** Dynamically allocate Pinecone namespaces per user session ID, preventing cross-tenant document context leaks.

Report generated automatically by Antigravity AI - QuickRAG Technical System Audit